



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

RISC-V Control Flow



[Warmup] lab03 Exercise 4 Hint 2

"the index is stored in t0 and the pointer to the array is stored in t1."

```
1 slli   t2 t0 2
2 add    t2 t2 t1
3 lw     t3 0(t2)
```

0b0...000010
0b0...001000

What does each instruction do?

See [green card](#)!



t0	2
t1	0x100
t2	0x108
t3	0x0000ABCD

	+3	+2	+1	+0
0x100	00	00	05	00
0x104	12	34	56	78
0x108	00	00	AB	CD
0x10c	00	12	EF	00
0x110	FA	CE	FA	CE

Yan, SP26



[Review] Register Conventions

- RISC-V **register convention** gives common **register names** to each register. This convention helps cleaner, faster assembly instructions.
 - **Temporary registers** t_0, t_1, t_2
 - **Stack pointer** sp : (yes, that one!) start of a usable block of memory on the stack

#	Name	Description
x0	zero	Constant 0
x1	ra	<i>Return Address</i>
x2	sp	Stack Pointer
x3	gp	Global Pointer
x4	tp	Thread Pointer
x5	t0	<i>Temporary Registers</i>
x6	t1	
x7	t2	

See [green card!](#)

SP26



[at home] Bitwise Ops

Understand why:

- Bitwise right shift has both arithmetic and logical (`sra`, `sr1`)
- Shift-left only needs one (`sll`)
- `not` is a pseudoinstruction for `xori` with `-1`

<https://notes.cs61c.org/content/misc/rv32i-green-card/>



Control Flow, Program Counter Register

- Lab 03 array review
- [at home] Bitwise Ops
- Control Flow, pc register
- Conditional Branches
- Loops



A Bit about Machine Programs

So far:

- Program values are stored in registers.
- Assembly instructions:
 - operate on registers
 - load/store values between registers and memory.

Each RISC-V assembly instruction:

- is stored as machine code, i.e., bits
- is **word-length** (32 bits)
- is **stored in memory** (text segment)

`slli x12 x10 0x10`

assembly instruction

(more
later)

`0x01051613`

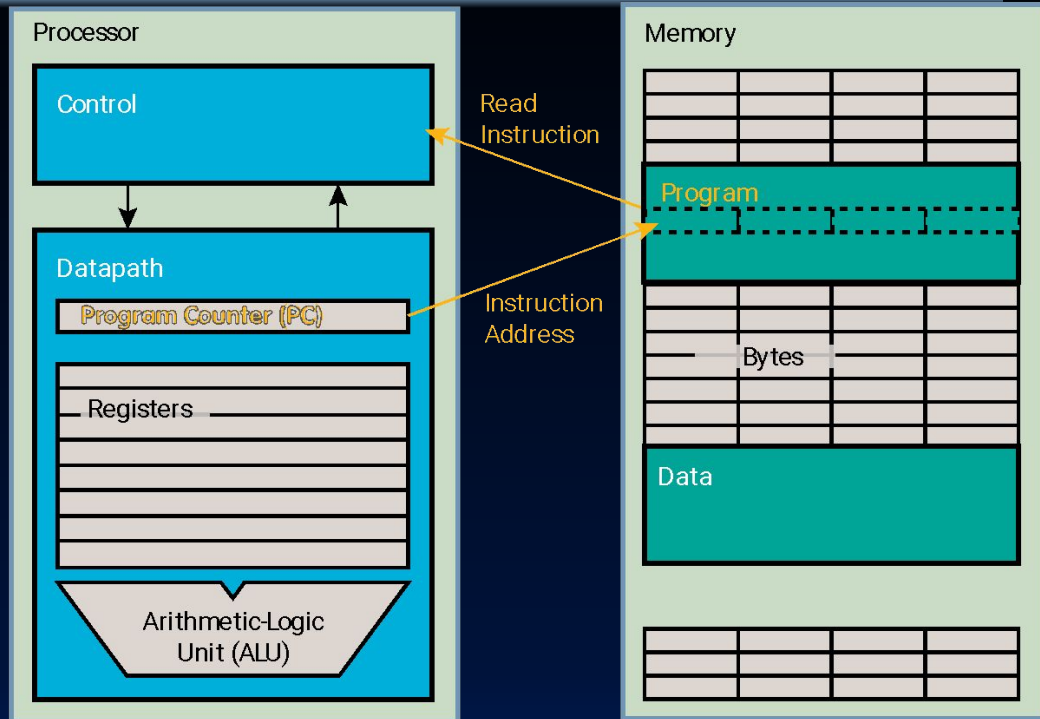
machine code



The Program Counter Register

RV32I Unprivileged ISA, 2.1:

There is one additional unprivileged register: the **program counter** **pc** holds the address of the current instruction.





Program Counter in action

pc ~~0x0000 0008~~
0x0000 000c



```
0x0  ...  
0x4  .. # li x10 0x34FF  
0x8  slli x12 x10 0x10  
0xc  srli x12 x12 0x08  
0x10 and x12 x12 x10
```

Program execution:

- Fetch instruction from memory using PC
- Execute instruction
- Update PC** to next instruction

x10

0x0000 34FF

x12

~~0xABCD ABCD~~

0x34FF 0000

By default, the PC is updated to PC + 4 bytes ("next sequential instruction") **unless otherwise stated by the instruction itself.**





Conditional Branches

- Lab 03 array review
- [at home] Bitwise Ops
- Control Flow, pc register
- Conditional Branches
- Loops

Control Flow in C

- In C, run code one line at a time.
 - **Control flow** syntax creates structures that “**jump**” to other non-sequential statements.
 - if, if-else, if-else-elif, etc.
 - For loops
 - While loops
 - Function calls (more later)

```
1  int foo(n) {  
2      int a = 5;  
3      if (n > 5) {  
4          a += 1;  
5      }  
6      return a+n;  
7  }  
...  
10 line;  
11 foo(5);  
12 line;
```

conditional jump

(otherwise)

unconditional jump

Control instructions in RISC-V adjust the program counter to **conditionally/unconditionally jump** to other instructions.



RISC-V Jumps

- In RISC-V, a **jump** is a change in control flow.
 - The program counter is updated to NOT the next instruction as is the default (RV32I: current +4 bytes), but another instruction.
- **Unconditional jump instruction:** `j Label`
 - Jump to the instruction specified by the label `Label`
 - A pseudoinstruction (more next time)
- Conditional jump, i.e., **branch instruction**
 - General format: `bxx rs1 rs2 Label`
 - Jumps to the specified `Label` **only if the condition is met**.
 - Otherwise (the condition is not met), just move to next line.

Branch if EQual, Branch if Not Equal

- **B**branch if **eq**ual

```
beq rs1 rs2 Label  
if (R[rs1] == R[rs2])  
    PC = <addr of Label>
```

- Means:

- Check if register values **are equal**: $R[rs1] == R[rs2]$
- If true, then go to statement labeled Label
- Otherwise, go to next statement ($PC = PC + 4$)

- **B**branch if **not e**qual

```
bne rs1 rs2 Label  
if (R[rs1] != R[rs2])  
    PC = <addr of Label>
```

- Means:

- Check if register values **are not equal**: $R[rs1] != R[rs2]$
- If true, then go to statement labeled Label
- Otherwise, go to next statement ($PC = PC + 4$)

Example 1: beq or bne?

Assume the following mapping of
C variables $\leftrightarrow$ registers:

x	y	z	i	j
x10	x11	x12	x13	x14

```
if (i == j) {  
    x = y + z;  
} else {  
    x = y - z;  
}  
// ...
```

A.

```
beq x13 x14 If  
sub x10 x11 x12  
j End  
If: add x10 x11 x12  
End: # ...
```

B.

```
bne x13 x14 Else  
add x10 x11 x12  
j End  
Else: sub x10 x11 x12  
End: # ...
```





Example 1: beq or bne?

Assume the following mapping of C variables $\leftrightarrow$ registers:

x	y	z	i	j
x10	x11	x12	x13	x14

```
if (i == j) {  
    x = y + z;  
} else {  
    x = y - z;  
}  
// ...
```

A.

```
beq x13 x14 If  
sub x10 x11 x12  
j End  
If:  add x10 x11 x12  
End: # ...
```

B.

```
bne x13 x14 Else  
add x10 x11 x12  
j End  
Else: sub x10 x11 x12  
End:  # ...
```

Both valid!

...but B more closely reflects the original C structure, despite the negated branch condition.

Example 2: beq or bne?

Assume the following mapping of C variables $\leftrightarrow$ registers:

x	y	z	i	j
x10	x11	x12	x13	x14

```
if (i == j) {
    x = y + z;
}
// ...
```

A.

```
beq x13 x14 If
j End
If:  add x10 x11 x12
End: # ...
```

B.

```
bne x13 x14 End
add x10 x11 x12
End: # ...
```

Both valid! ...but...

For readable, efficient RISC-V programs, you may need to **negate** the branch condition!



RISC-V Control Flow Operations

Instruction	Name/description
<code>beq rs1 rs2 Label</code>	Branch if Equal
<code>bne rs1 rs2 Label</code>	Branch if Not Equal
<code>blt rs1 rs2 Label</code>	Branch if Less Than (signed) ($rs1 < rs2$)
<code>bge rs1 rs2 Label</code>	Branch if Greater or Equal (signed) ($rs1 \geq rs2$)
<code>bltu rs1 rs2 Label</code>	Branch if Less Than (unsigned)
<code>bgeu rs1 rs2 Label</code>	Branch if Greater Than or Equal (unsigned)
<code>j Label</code>	Unconditional jump

`bgt`, `bgtu`, `ble`, and `bleu` are **not** base RV32I instructions!

- Pseudoinstructions
- At home: Create these by reversing inputs of existing instructions.



Loops

- Lab 03 array review
- [at home] Bitwise Ops
- Control Flow, pc register
- Conditional Branches
- Loops



Loops in C → Loops in Assembly

- Several types of loops in C:
 - while
 - for
 - do-while
- (CS 61A) Each can be rewritten as the other two.
- Once we learn how to write conditional branches with one loop, we can apply the same branching method to all loops.
 - Rethink C code using goto statements



Reducing C with goto: While

```
while(cond) {  
    line1;  
    line2;  
}  
line3;
```

```
Loop:  
    if(!cond) goto AfterLoop;  
    line1;  
    line2;  
    goto Loop;
```

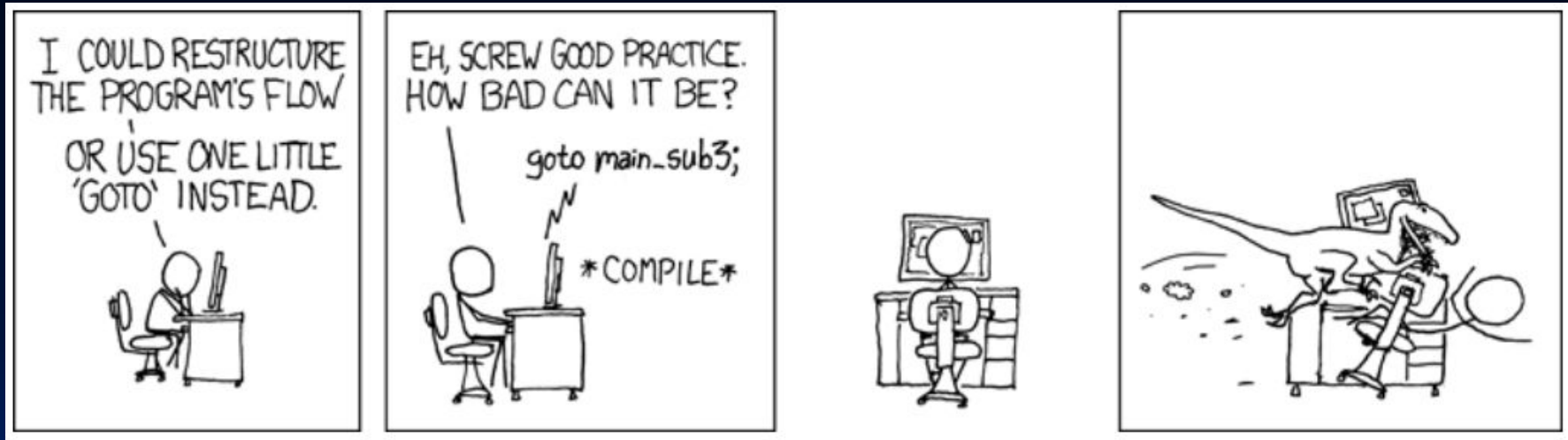
```
AfterLoop:  
    line3;
```

Labels are not C statements/RV instructions.
In RV, labels are names are bound to specific
instruction addresses.

NEVER write your C code with goto



We suggest never writing C code with **goto**. However, understanding how to translate into goto helps to translate into assembly/RISC-V.



[xkcd](#), [explainxkcd](#)

lab03: Fibonacci Tutorial

$$F_0 = 0, \quad F_1 = 1,$$

and

$$F_n = F_{n-1} + F_{n-2}$$

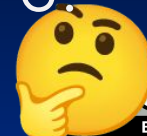
for $n > 1$.

```
1      # ... setup ...
2  fib:
3      beq t3 x0 finish
4      add t2 t1 t0
5      mv  t0 t1
6      mv  t1 t2
7      addi t3 t3 -1
8      j  fib
9  finish:
10     # ... print ...
```

```
1      ...
2  int curr_fib = 0;
3  int next_fib = 1;
4  int new_fib;
5  for (int i = 12; i > 0; i--) {
6      new_fib = \
7          curr_fib + next_fib;
8      curr_fib = next_fib;
9      next_fib = new_fib;
10 }
11 ...
```

fib(12) = 144.

1. Map C variables to registers.
2. Why beq and not bgt pseudo?
3. How does j fib update PC?



Exercise (1/9): Rewrite our C for loop

(original for loop)

```
int arr[20];  
... // fill arr with data  
int sum = 0;  
for (int i=0; i<20; i++) {  
    sum += arr[i];  
} // ...
```

(rewrite with while loop)

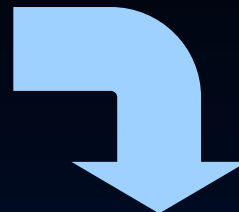
```
int arr[20];  
...  
int sum = 0;  
int i = 0;  
while (i < 20) {  
    sum += arr[i];  
    i++;  
} // ...
```

(rewrite with goto)

```
int arr[20];  
...  
int sum = 0;  
int i = 0;  
Loop: if(i >= 20) goto End  
    sum += arr[i];  
    i++;  
    goto Loop  
End: // ...
```

Exercise (2/9): RISC-V Code

```
int arr[20];
... // fill arr with data
int sum = 0;
for (int i=0; i<20; i++) {
    sum += arr[i];
} // ...
```



```

add    x9    x8    x0
add    x10   x0    x0
add    x11   x0    x0
addi   x13   x0    20
Loop:  bge    x11   x13   End
      lw     x12   0(x9)
      add    x10   x10   x12
      addi   x9     x9     4
      addi   x11   x11    1
      j      Loop
End:   ...
```



```

(rewrite  int arr[20];
with      ...
goto)    int sum = 0;
         int i = 0;
         Loop: if(i >= 20) goto End
               sum += arr[i];
               i++;
               goto Loop
         End:  // ...
```

Exercise (3/9): Map C expressions to registers

```
int arr[20];
... // fill arr with data
int sum = 0;
for (int i=0; i<20; i++) {
    sum += arr[i];
} // ...
```

```

      add    x9    x8    x0
      add    x10   x0    x0
      add    x11   x0    x0
      addi   x13   x0    20
Loop: bge    x11   x13   End
      lw     x12   0(x9)
      add    x10   x10   x12
      addi   x9    x9    4
      addi   x11   x11   1
      j      Loop
End:  ...
```

1. x8
2. x11
3. x9
4. x12
5. x10
6. x13

- A. sum
- B. i
- C. 20
- D. arr[0]
- E. &arr[0]
- F. arr[i]
- G. &arr[i]

Suppose x8 has the address of arr. Map the remaining registers to C variables/expressions.



Exercise (4/9): Assume x8

x8: &arr[0]

```

int arr[20];
... // fill arr with data
int sum = 0;
for (int i=0; i<20; i++) {
    sum += arr[i];
} // ...
    
```

```

                                add    x9    x8    x0
                                add    x10   x0     x0
                                add    x11   x0     x0
                                addi   x13   x0     20
Loop: bge    x11  x13  End
      lw     x12  0(x9)
      add    x10  x10  x12
      addi   x9   x9   4
      addi   x11  x11  1
      j     Loop
End:    ...
    
```

1. x8

2. x11

3. x9

4. x12

5. x10

6. x13

A. sum

B. i

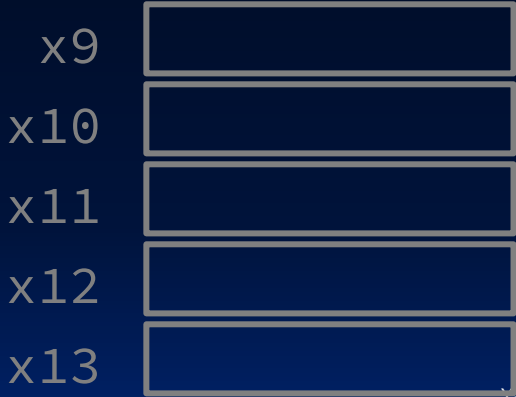
C. 20

D. arr[0]

E. &arr[0]

F. arr[i]

G. &arr[i]



Exercise (5/9): x11

x8: &arr[0]

```

int arr[20];
... // fill arr with data
int sum = 0;
for (int i=0; i<20; i++) {
    sum += arr[i];
} // ...

```

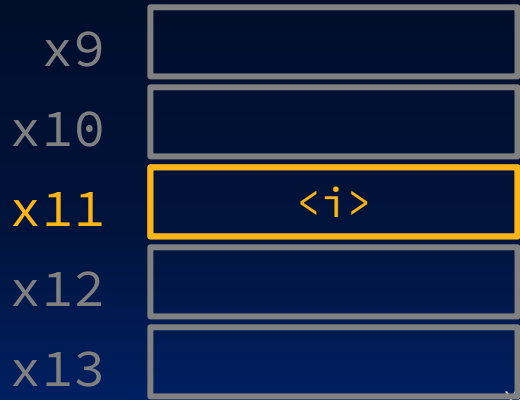
```

      add    x9    x8    x0
      add    x10   x0    x0
  Loop: add    x11   x0    x0  # i=0
      addi   x13   x0    20
      bge    x11   x13   End
      lw     x12   0(x9)
      add    x10   x10   x12
      addi   x9    x9    4
      addi   x11   x11   1  # i++
      j      Loop
End:   ...

```

1. x8
2. x11
3. x9
4. x12
5. x10
6. x13

A. sum
B. i
C. 20
D. arr[0]
E. &arr[0]
F. arr[i]
G. &arr[i]



Exercise (6/9): x9

x8: &arr[0]

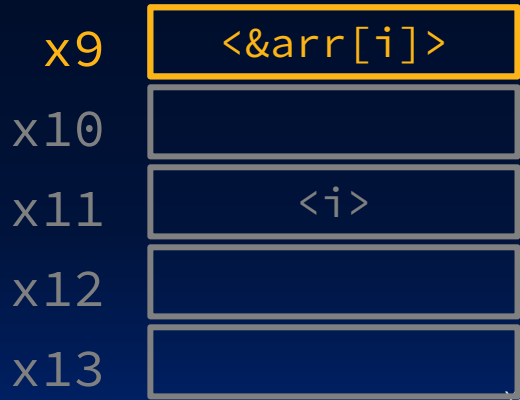
```
int arr[20];
... // fill arr with data
int sum = 0;
for (int i=0; i<20; i++) {
    sum += arr[i];
} // ...
```

```

add    x9    x8    x0    # &arr[0]
add    x10   x0    x0
add    x11   x0    x0    # i=0
addi   x13   x0    20
Loop:  bge    x11  x13  End
      lw     x12   0(x9)
      add    x10  x10  x12
      addi   x9    x9    4    # &arr[i+1]
      addi   x11  x11    1    # i++
      j     Loop
End:  ...
```

1. x8
2. x11
3. x9
4. x12
5. x10
6. x13

A. sum
B. i
C. 20
D. arr[0]
E. &arr[0]
F. arr[i]
G. &arr[i]



Exercise (7/9): x10, x12

x8: &arr[0]

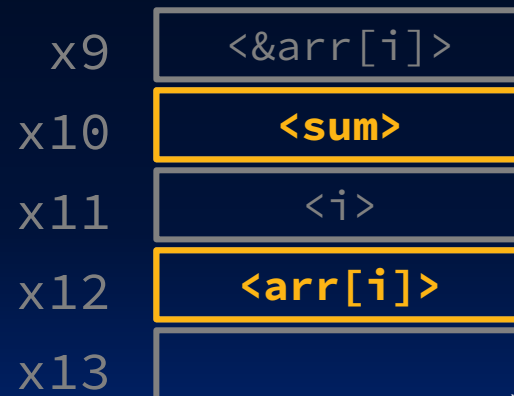
```
int arr[20];
... // fill arr with data
int sum = 0;
for (int i=0; i<20; i++) {
    sum += arr[i];
} // ...
```

```

add    x9    x8    x0    # &arr[0]
add    x10   x0    x0    # sum = 0
add    x11   x0    x0    # i=0
addi   x13   x0    20
Loop:  bge    x11  x13  End
      lw     x12  0(x9)
      add    x10  x10  x12  # sum+=arr[i]
      addi   x9    x9    4  # &arr[i+1]
      addi   x11   x11   1  # i++
      j      Loop
End:  ...
```

1. x8
2. x11
3. x9
- 4. x12**
- 5. x10**
6. x13

- A.** sum
- B. i
- C. 20
- D. arr[0]
- E. &arr[0]
- F.** arr[i]
- G. &arr[i]



Exercise (8/9): x13

x8: &arr[0]

```
int arr[20];
... // fill arr with data
int sum = 0;
for (int i=0; i<20; i++) {
    sum += arr[i];
} // ...
```

```

add    x9    x8    x0    # &arr[0]
add    x10   x0    x0    # sum = 0
add    x11   x0    x0    # i=0
addi   x13   x0    20
Loop:  bge    x11   x13 End    # i >= 20 → jump
      lw     x12   0(x9)
      add    x10   x10   x12   # sum+=arr[i]
      addi   x9    x9     4    # &arr[i+1]
      addi   x11   x11    1    # i++
      j      Loop
End:   ...
```

- 6.** x13 **C.** 20
- **bge compares two registers;** the immediate 20 must therefore be in register x13
 - bge exits loop if true.
 - Otherwise if false, then i < 20; continue in loop

x9	<&arr[i]>
x10	<sum>
x11	<i>
x12	<arr[i]>
x13	20

ran, SP26

Exercise (9/9): Full Code

x8: &arr[0]

```
int arr[20];
... // fill arr with data
int sum = 0;
for (int i=0; i<20; i++) {
    sum += arr[i];
} // ...
```

- **bge compares two registers;** the immediate 20 must therefore be in register x13
- bge exits loop if true.
- Otherwise if false, then $i < 20$; continue in loop

```

add    x9    x8    x0    # &arr[0]
add    x10   x0    x0    # sum = 0
add    x11   x0    x0    # i=0
addi   x13   x0    20
Loop:  bge    x11   x13   End    # i >= 20 → jump
      lw     x12   0(x9)
      add    x10   x10   x12    # sum+=arr[i]
      addi   x9     x9     4    # &arr[i+1]
      addi   x11   x11    1    # i++
      j      Loop
End:   ...
```

x9	<&arr[i]>
x10	<sum>
x11	<i>
x12	<arr[i]>
x13	20

ran, SP26

[Summary] RISC-V So Far

- Arithmetic
 - add
 - sub
 - and
 - or
 - xor
 - sll
 - srl
 - sra
 - mv (pseudo)
- Immediate
 - addi
 - andi
 - ori
 - xori
 - slli
 - srli
 - srai
 - li (pseudo)
- Loads/Stores
 - lw
 - lb
 - lbu
 - sw
 - sb
- Branches/Jumps
 - beq
 - bne
 - bge
 - blt
 - bgeu
 - bltu
 - j (pseudo)